

发现易受攻击程序可以通过观察程序源码,追踪程序处理特大型输入时的执行情况,或者使用特定工具,如第七部分中讨论的 *fuzzing*,可以自动发现潜在的易受攻击的程序。攻击者会对其导致的内存数据毁坏做什么处理相当多样化,这要取决于被改写的数据内容。

7.5.2 预防缓冲区溢出

发现并利用堆栈缓冲区溢出并不困难。在过去几十年中利用其攻击成功的显著效果已经清楚地说明了这点。因此非常有需要来保护系统以免受缓冲区攻击,可以预防或者至少要能够检测并且阻止此类攻击。广义上保护对策分为两类:

- 编译时防御系统,目的是强化系统以抵制潜伏于新程序中的恶意攻击。
- 运行时防御系统,目的是检测并中止现有程序中的恶意攻击。

尽管合适的防御系统已经出现几十年了,但是大量现有的脆弱的软件和系统阻碍了它们的部署。因此运行时防御有趣的地方是它能够部署在操作系统中,可以更新,并能为现有的易受攻击的程序提供保护。

7.6 小结

内存管理是操作系统中最重要、最复杂的任务之一。内存管理把内存看做是一个资源,可以分配给多个活动进程,或者由多个活动进程共享。为了有效地使用处理器和 I/O 设备,需要在内存中保留尽可能多的进程。此外,程序员在进行程序开发时最好能不受程序大小的限制。

内存管理的基本工具是分页和分段。采用分页技术,每个进程被划分成相对较小的、大小固定的页。采用分段技术可以使用大小不同的块。还可以在一个单独的内存管理方案中把分页技术和分段技术结合起来使用。

7.7 推荐读物

由于分区技术已经被虚拟内存所取代,因而大多数操作系统书籍中仅对分区技术进行了粗略的介绍,[MILE92]提供了最完全、最有趣的介绍。[KNUT97]给出了关于分区策略最全面的讨论。

有关链接和加载的主题包含在许多关于程序开发、计算机体系结构和操作系统方面的书籍中,[BECK97]中对其进行了非常详细的介绍。[CLAR98]中也有很好的论述。[LEVI00]给出了对其在实践中的应用进行了周详的讨论,其中包含有大量的操作系统示例。

BECK97 Beck, L. *System Software*. Reading, MA: Addison-Wesley, 1997.

CLAR98 Clarke, D., and Merusi, D. *System Software Programming: The Way Things Work*. Upper Saddle River, NJ: Prentice Hall, 1998.

KNUT97 Knuth, D. *The Art of Computer Programming, Volume I: Fundamental Algorithms*, 2nd Ed. Reading, MA: Addison-Wesley, 1997.

LEVI00 Levine, J. *Linkers and Loaders*. San Francisco: Morgan Kaufmann, 2000.

MILE92 Milenkovic, M. *Operating Systems: Concepts and Design*. New York: McGraw-Hill, 1992.

7.8 关键术语、复习题和习题

关键术语

绝对加载	固定分区	逻辑组织	物理组织
伙伴系统	页框	内存管理	保护
压缩	内部碎片	页面	相对地址
动态链接	链接编辑程序	页表	可重定位加载
动态分区	链接	分页	重定位
动态运行时加载	加载	分区技术	分段
外部碎片	逻辑地址	物理地址	共享

复习题

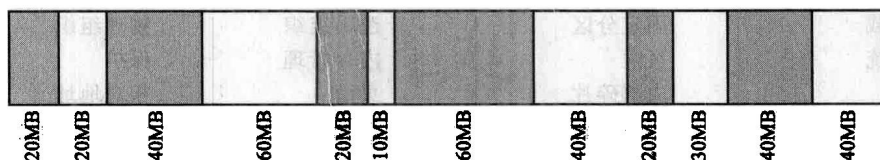
- 7.1 内存管理需要满足哪些需求？
- 7.2 为什么需要重定位进程的能力？
- 7.3 为什么不可能在编译时实施内存保护？
- 7.4 允许两个或多个进程访问内存某一特定区域的原因是什么？
- 7.5 在固定分区方案中，使用大小不等的分区有什么好处？
- 7.6 内部碎片和外部碎片有什么区别？
- 7.7 逻辑地址、相对地址和物理地址间有什么区别？
- 7.8 页和页框之间有什么区别？
- 7.9 页和段之间有什么区别？

习题

- 7.1 2.3 节中列出了内存管理的 5 个目标，7.1 节中列出了 5 种需求。请说明它们是一致的。
- 7.2 假设一个多道程序系统采用固定分区的内存管理策略，分区的大小分别为 Part0=20K，Part1=30K，Part2=30K，和 Part3=50K。同时假设下表中的进程需要调度进内存。假定这些进程都是（大致上）按照它们的进程号的顺序在同一时刻（时间 0）到达的。表格中的时间一栏表示这个进程需要分配一个分区来完成的时间，包含了完成该进程的全部时间（例如，交换时间，CPU 时间等等。）假设一个进程一旦被分配内存，该进程会一直驻留在内存中直到它完成为止（没有交换发生）。
 - a) 如果每一个进程都被分配最小的分区，且能够满足它的需求（在某些教材中被称为最佳适应分配策略），那么系统需要多长时间执行完表中所有进程。解释你的答案。
 - b) 如果每一个进程都被分配最小的空闲分区，且能够满足它的需求（在某些教材中被称为最佳可得分配策略），那么系统需要多长时间执行完表中全部的进程。解释你的答案。

进 程	最大内存需求(K)	时间(ms)
1	18	10
2	29	5
3	10	20
4	33	15
5	14	15
6	49	10

- 7.3 说明下面每一种内存分配方案平均情况下的内部碎片的大小。回答尽可能详细。
 - a) 动态分区
 - b) 简单分页
 - c) 伙伴系统
- 7.4 为实现动态分区中的各种放置算法（见 7.2 节），内存中必须保留一个空闲块列表。分别讨论最佳适配、首次适配、下次适配三种方法的平均查找长度。
- 7.5 动态分区的另一种放置算法是最差适配，在这种情况下，当调入一个进程时，使用最大的空闲存储块。该方法与最佳适配、首次适配、下次适配相比，优点和缺点各是什么？它的平均查找长度是多少？
- 7.6 如果使用动态分区方案，下图所示为在某个给定的时间点的内存结构：



阴影部分为已经被分配的块；空白部分为空闲块。接下来的三个内存需求分别为 40MB、20MB 和 10MB。分别使用如下几种放置算法，指出给这三个需求分配的块的起始地址。

- a) 首次适配
- b) 最佳适配
- c) 下次适配 (假定最近添加的块位于内存的开始)
- d) 最差适配

7.7 假设你的计算机操作系统使用伙伴系统来进行内存管理。初始化的时候系统有 1 兆 (1024KB) 内存块是空闲的, 这些内存块起始地址为 0。参考图 7.6 中的表达方式, 利用一组数字来说明每一次内存申请和释放之后的结果。

- A: 申请 25KB
- B: 申请 500KB
- C: 申请 60KB
- D: 申请 100KB
- E: 申请 30KB
- 释放 A
- F: 申请 20KB

在给 F 进程分配内存之后, 系统中存在多少个内部碎片?

7.8 考虑一个伙伴系统, 在当前分配下的一个特定块地址为 011011110000。

- a) 如果块大小为 4, 它的伙伴的二进制地址为多少?
- b) 如果块大小为 16, 它的伙伴的二进制地址为多少?

7.9 令 $\text{buddy}_k(x)$ 为大小为 2^k 、地址为 x 的块的伙伴的地址, 写出 $\text{buddy}_k(x)$ 的通用表达式。

7.10 Fibonacci 序列定义如下:

$$F_0=0, \quad F_1=1, \quad F_{n+2}=F_{n+1}+F_n, \quad n \geq 0$$

- a) 这个序列可以用于建立伙伴系统吗?
- b) 该伙伴系统与本章介绍的二叉伙伴系统相比, 有什么优点?

7.11 在程序执行期间, 每次取指令后处理器把指令寄存器的内容 (程序计数器) 增加一个字, 但如果遇到会导致在程序中其他地址继续执行的跳转或调用指令, 处理器将修改这个寄存器的内容。现在考虑图 7.8, 关于指令地址有两种选择:

- 在指令寄存器中保存相对地址, 并把指令寄存器作为输入进行动态地址转换。当遇到一次成功的跳转或调用时, 由这个跳转或调用产生的相对地址被装入到指令寄存器中。
 - 在指令寄存器中保存绝对地址。当遇到一次成功的跳转或调用时, 采用动态地址转换, 其结果保存在指令寄存器中。
- 哪种方法更好?

7.12 在一个 32 位的机器上, 假设把逻辑地址分为 8 位 6 位 6 位 12 位四个部分。换句话说, 系统使用 3 级页表, 其中第一个 8 位是第一级, 后面的 6 位是第二级, 以此类推。在这个系统中, 用 6 位表示页号。假设内存是按照字节访问的。

- a) 该系统中页的大小是多少?
- b) 能够分配给一个进程的最大的页面个数是多少?
- c) 逻辑地址空间最大是多少?
- d) 物理内存的大小?

7.13 分页系统中的虚拟地址 a 相当于一个 (p, w) 对, 其中 p 是页号, w 是在页中的字节号。令 z 是一页中的字节总数, 请给出 p 和 w 关于 z 和 a 的函数。

7.14 在一个简单分段系统中, 包含如下段表:

起始地址	长度 (字节)
660	248
1752	442
222	198
996	604

对如下的每一个逻辑地址, 确定其对应的物理地址或者说明段错误是否会发生:

- a) 0, 198
- b) 2, 156
- c) 1, 530

d) 3, 444

e) 0, 222

7.15 在内存中, 存在连续的段 $S_1, S_2, \dots, S_n$ 按其创建顺序依次从一端放置到另一端, 如下图所示:



当段 S_{n+1} 被创建时, 尽管 $S_1, S_2, \dots, S_n$ 中的某些段可能已经被删除, 段 S_{n+1} 仍被立即放置在段 S_n 之后。当段 (正在使用或已被删除) 和洞之间的边界到达内存的另一端时, 压缩正在使用的段。

a) 说明花费在压缩上的时间 F 遵循以下不等式:

$$F \geq \frac{1-f}{1+kf}, \text{ 其中 } k = \frac{t}{2s} - 1$$

其中, s 表示段的平均长度 (以字为单位);

t 表示段的平均生命周期, 即内存访问次数;

f 表示在平衡条件下, 未使用的内存部分的比例。

提示: 计算边界在内存中移动的平均速度, 并假设复制一个字至少需要两次内存访问。

b) 当 $f=0.2, t=1000, s=50$ 时, 计算 F 。

附录 7A 加载和链接

创建活动进程的第一步是把程序装入内存, 并创建一个进程映像 (如图 7.15 所示)。图 7.16 描述了大多数系统中的一种典型场景。应用程序由许多已编译过的或汇编过的模块组成, 这些模块以目标代码的形式存在, 并被链接起来以解析模块间的任何访问和对库例程的访问。库例程可以被合并到程序中, 或作为操作系统在运行时提供的共享代码被访问。这个附录将着重总结链接器和加载器的主要特征。为了表达得更清楚, 首先描述只涉及一个程序模块时的加载任务, 这时不需要链接。

加载

在图 7.16 中, 加载器把加载的模块放置在内存中从 x 开始的位置。在加载这个程序的过程中, 必须满足图 7.1 中所描述的寻址需求。一般而言, 可以采用三种方法: 绝对加载、可重定位加载、动态运行时加载。

绝对加载

绝对加载器要求一个给定的加载模块总是被加载到内存中的同一个位置。因此, 在提供给加载器的加载模块中, 所有的地址访问必须是确定的, 或者说是绝对的内存地址。例如, 如果图 7.16 中的 x 是 1024 位置, 则加载模块中的第一个字在内存中的地址为 1024。

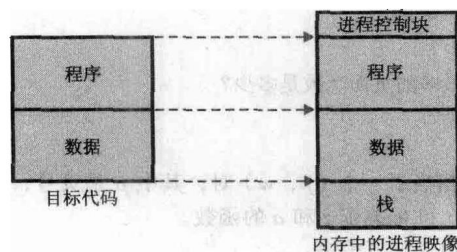


图 7.15 加载的功能

给程序中的内存访问指定具体的地址值既可以由程序员完成, 也可以在编译时或者汇编时完成, 见表 7.3a。这种方法存在许多缺点: 首先, 程序员必须知道在内存中放置模块时预定的分配策略。其次, 如果在程序的模块体中进行了任何涉及插入或删除的修改, 则所有地址都需要更改。因此, 更可取的方法是允许用符号表示程序中的内存访问, 然后在编译或者汇编时解析这些符号引用, 如图 7.17 所示。对指令或数据项的引用最初被表示成一个符号。在准备输入到一个绝对加载器的模块时, 汇编器或编译器将把所有这些引用转换成具体地址。在图 7.17b 的例子中, 模块被加载到从 1024 位置开始的位置。

可重定位加载

在加载之前就把内存访问绑定到具体的地址的缺点是, 这会使得加载模块只能放置到内存中的一个区

域。但是,当多个程序共享内存时,不可能事先确定哪块区域用于加载哪个特定的模块,最好是在加载时确定。因此,需要一个可以被分配到内存中任何地方的加载模块。

为满足这个新需求,汇编器或编译器不产生实际的内存地址(绝对地址),而是相对于某些已知点的地址,如相对于程序的起点,这个技术如图 7.17c 所示。加载模块的起点被指定为相对地址 0,模块中的所有其他内存访问都用与该模块起点的相对值来表示。

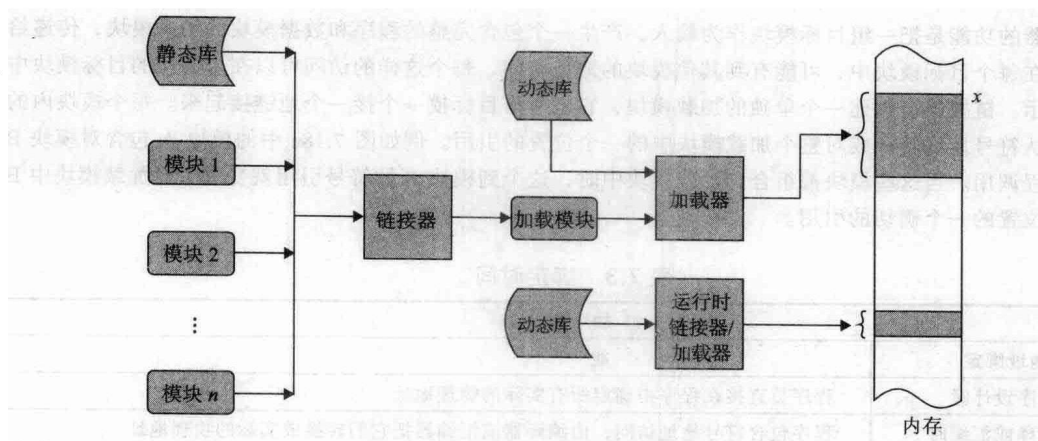


图 7.16 连接和加载场景

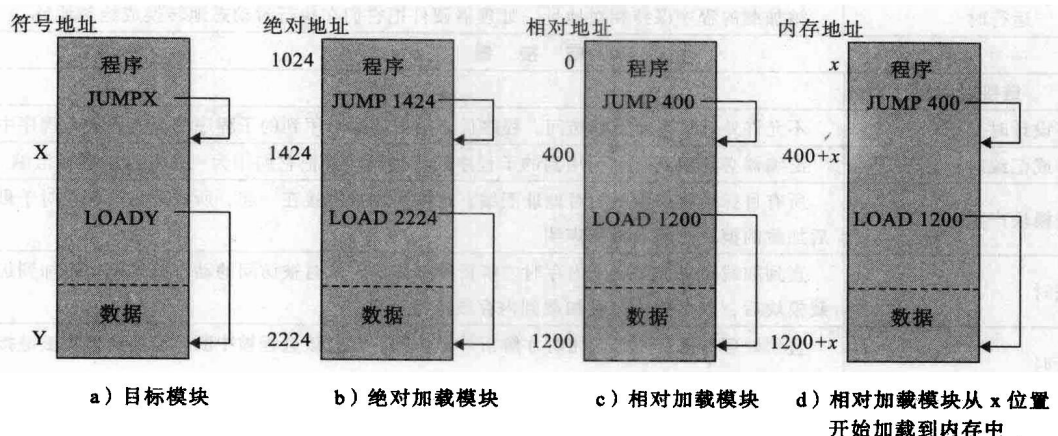


图 7.17 绝对和可重定位加载模块

既然所有内存访问都以相对的形式表示,加载器就可以很容易地把模块放置在期望的位置。如果该模块要加载到从 x 位置开始的地方,则当加载器把该模块加载到内存中时,只需要简单地给每个内存访问都加上 x 。为完成这个任务,加载模块必须包含一些需要告诉加载器的信息,如地址访问在哪里、它们如何被解释(通常相对于程序的起点,但也可能相对于程序中的某些其他点,如当前位置)。由编译器或汇编器准备这些信息,通常称这些为重定位地址库。

动态运行时加载

可重定位加载器是非常普遍的,并且相对于绝对加载器具有明显的优点。但是,在多道程序设计环境中,即使不依赖于虚拟内存,可重定位的加载方案仍是不够的。由于需要把进程换入或换出内存,以增大处理器的利用率。为最大程度地利用内存,又希望能在不同的时刻把一个进程映像换回到不同的位置。这样程序被加载后,可能被换出到磁盘,然后又被换回到内存中不同的位置。如果在开始加载的时候,内存访问就被绑定到绝对地址,那么前面提到的情况将是不可能实现的。

一种替代的方案是在运行时真正在使用某个绝对地址时再计算它。为达到这个目的,加载模块被加载到内存中时,它的所有内存访问都以相对形式表示,如图 7.17c 所示,一条指令只有在真正被执行时才计

算绝对地址。为确保该功能不会降低性能，这些工作必须由特殊的处理器硬件完成，而不是用软件实现（见 7.2 节）。

动态地址计算提供了完全的灵活性。一个程序可以加载到内存中的任何区域，程序的执行可以中断，程序还可以被换出内存，以后再换回到不同的位置。

链接

链接器的功能是把一组目标模块作为输入，产生一个包含完整的程序和数据模块的加载模块，传递给加载器。在每个目标模块中，可能有到其他模块的地址访问，每个这样的访问可以在未链接的目标模块中用符号表示。链接器会创建一个单独的加载模块，它把所有目标模块一个接一个地链接起来。每个模块内的引用必须从符号地址转换成对整个加载模块中的一个位置的引用。例如如图 7.18a 中的模块 A 包含对模块 B 的一个过程调用。当这些模块都组合到加载模块中时，这个到模块 B 的符号引用就变成了对加载模块中 B 的入口点位置的一个确切的引用。

表 7.3 绑定时间

a) 加 载 器	
地址绑定	功 能
程序设计时	程序员直接在程序中确定所有实际的物理地址
编译或汇编时	程序包含符号地址访问，由编译器或汇编器把它们转换成实际的物理地址
加载时	编译器或汇编器产生相对地址，加载器在加载程序中把它们转换成实际的绝对地址
运行时	被加载的程序保持相对地址，处理器硬件把它们在运行时动态地转换成绝对地址
b) 链 接 器	
链接时间	功 能
程序设计时	不允许外部程序或数据访问。程序员必须把所有引用到的子程序的源代码放入程序中
编译或汇编时	汇编器必须取到每个引用到的子程序的源代码，并把它们作为一个部件来进行汇编
加载模块产生时	所有目标模块都使用相对地址汇编。这些模块被链接在一起，所有的访问都相对于最后加载的模块的地点重新声明
加载时	直到加载模块被加载到内存时才解析外部访问，此时被访问的动态链接模块附加到加载模块后，整个软件包被加载到内存或虚拟内存
运行时	直到处理器执行外部调用时才解析外部访问，此时该进程被中断，需要的模块被链接到调用程序中

链接编辑程序

地址链接的性质取决于链接发生时要创建的加载模块的类型，见表 7.3b。通常情况下需要可重定位的加载模块，然后链接按以下方式完成。每个已编译过或汇编过的目标模块连同相对于该目标模块开始处的引用一同被创建。所有这些模块，连同相对于该加载模块起点的所有引用，一起放进一个可重定位的加载模块中。该模块可以作为可重定位加载或动态运行时加载的输入。

产生可重定位加载模块的链接器通常称做链接编辑程序。图 7.18 说明了链接编辑程序的功能。

动态链接器

像加载一样，可能推迟某些链接功能。动态链接（dynamic linking）是指把某些外部模块的链接推迟到创建加载模块之后。因此，加载模块包含到其他程序的未解析的引用，这些引用可以在加载时或运行时被解析。

对于加载时的动态链接（包括图 7.16 中上面的动态库），分为如下步骤。将被加载的加载模块（应用模块）读入内存。应用模块中到一个外部模块（目标模块）的任何引用都导致加载程序查找目标模块，加载它，并把这些引用修改成相对于应用程序模块开始处的相对地址。该方法比静态链接有以下优点：

- 能够更容易地并入已改变或已升级了的目标模块，如操作系统工具，或者某些其他的通用例程。

而对于静态链接，这类支持模块的变化将需要重新链接全部应用程序模块。除了静态链接比较低

效以外,在某些情况下甚至不可能使用静态链接。例如,在个人计算机领域,大多数商用软件以加载模块的形式发布,而不会公布源程序和目标程序。

- 在动态链接文件中的目标代码为自动代码共享铺平了道路。因为操作系统加载并链接了该代码,所以可以识别出有多个应用程序使用相同的目标代码。操作系统可以使用此信息,然后只加载目标代码的一个副本,并把这个被加载的目标副本链接到所有使用该目标代码的应用程序,而不是为每个应用程序都分别加载一个副本。
- 它使得独立软件开发人员可以更容易地扩展诸如 Linux 之类的广泛使用的操作系统的功能。开发人员可以设计出一个对各种应用程序都很有用的新功能,并把它包装成一个动态链接模块。

使用运行时动态链接时(包括图 7.16 中下面的动态库),某些链接工作被推迟到执行时。这样一些对目标模块的外部引用保留在被加载的程序中,当调用的模块不存在时,操作系统定位该模块,加载它,并把它链接到调用模块中,这些模块一般是共享的。在 Windows 环境下,这些模块叫做动态链接库(DLL)。也就是说,如果一个进程已经使用了动态链接共享模块,该模块就位于内存中,新的进程就可以简单地链接上已经加载好的模块了。

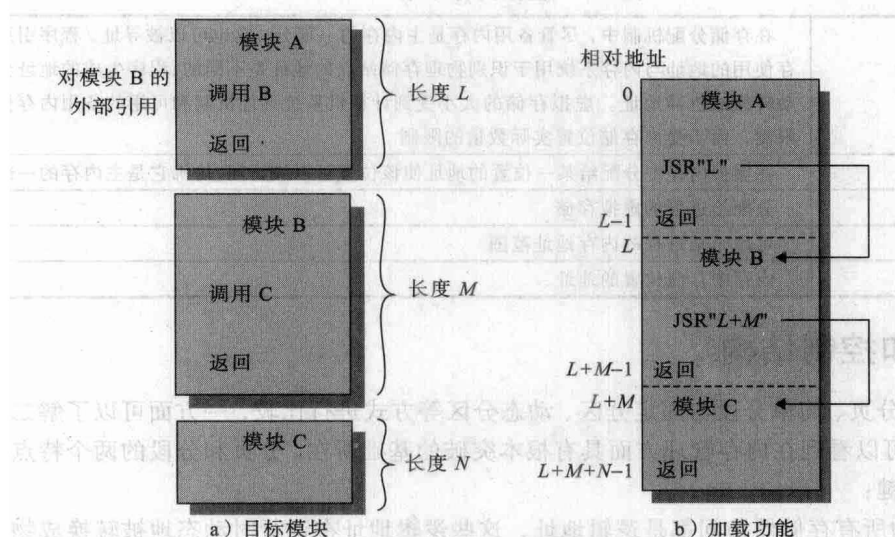


图 7.18 链接功能

使用 DLL 可能会导致一个问题,一般称之为 DLL 地狱(DLL Hell)。两个或更多的进程共享一个 DLL 模块,但是它们希望链接不同版本的模块时,就会发生 DLL 地狱问题。例如,一个应用软件或者系统功能可能会重新安装,因而带入了一个老的 DLL 版本文件。

虽然动态加载允许一个完整的加载模块到处移动,但是该模块的结构是静态的,在进程执行期间以及从一次执行到下一次执行都保持不变。在某些情况下,不可能在执行前确定需要哪个目标模块,事务处理应用程序就是这类情况的典型代表,如航空公司预订系统或银行业应用程序。需要根据事务的性质指定需要哪个程序模块,然后加载它并链接到主程序。使用动态链接器的优点是在程序单元被引用之前,不需要为它们分配内存空间,这种能力可用于支持分段系统。

还有另外一种改进:应用程序不需要知道可能会调用的所有模块名或入口点。例如,制表程序可能会和各种绘图仪一起工作,每个绘图仪都由不同的驱动软件包驱动,应用程序可以从另一个进程中得知或者从配置文件中查找到绘图仪的名字,这种改进允许应用程序的用户安装一个在编写该程序时并不存在的新绘图仪。

第 8 章 虚 拟 内 存

第 7 章介绍了分页和分段的概念，并分析了它们各自的缺点。我们从本章开始讨论虚拟内存。由于内存管理同处理器硬件和操作系统软件都有着紧密而复杂的关系，所以关于这方面的分析是非常复杂的。本章首先重点讲述虚拟内存的硬件特征，考虑使用分页、分段和段页式这三种情况，然后考虑操作系统中虚拟内存设施的设计问题。

表 8.1 给出了一些虚拟内存相关的定义。

表 8.1 虚拟内存术语

虚拟内存	在存储分配机制中，尽管备用内存是主内存的一部分，它也可以被寻址。程序引用内存使用的地址与内存系统用于识别物理存储站点的地址是不同的，程序生成的地址会自动转换成机器地址。虚拟存储的大小受到计算机系统寻址机制和可用的备用内存量的限制，而不受内存存储位置实际数量的限制
虚拟地址	在虚拟内存中分配给某一位置的地址使该位置可以被访问，仿佛它是主内存的一部分
虚拟地址空间	分配给进程的虚拟存储
地址空间	可用于某进程的内存地址范围
实地址	内存中存储位置的地址

8.1 硬件和控制结构

通过对简单分页、简单分段与固定分区、动态分区等方式进行比较，一方面可以了解二者的区别，另一方面可以看到在内存管理方面具有根本突破的基础所在。分页和分段的两个特点是取得这种突破的关键：

- 1) 进程中的所有存储器访问都是逻辑地址，这些逻辑地址在运行时动态地被转换成物理地址。这意味着一个进程可以被换入或换出内存，使得进程可以在执行过程中的不同时刻占据内存中的不同区域。
 - 2) 一个进程可以划分成许多块（页和段），在执行过程中，这些块不需要连续地位于内存中。动态运行时地址转换和页表或段表的使用使这一点成为可能。
- 如果具备前面的两个特点，那么在进程的执行过程中，该进程不需要所有页或所有段都在内存中。如果内存中保存着要取的下一条指令的所在块（段或页）以及将要访问的下一个数据单元的所在块，那么执行至少可以暂时继续下去。

现在考虑如何实现这一点。用术语“块”来表示页或段，取决于是采用分页还是分段机制。假设需要把一个新进程放入内存时，操作系统仅读取包含程序开始处的一个或几个块。进程执行中的任何时候都在内存中的部分被定义成进程的常驻集（resident set）。当进程执行时，只要所有的存储器访问都是要访问常驻集中的单元，执行就可以顺利进行；通过使用段表或页表，处理器总是可以确定是否如此。如果处理器需要访问一个不在内存中的逻辑地址，则产生一个中断，说明产生了内存访问故障。操作系统把被中断的进程置于阻塞状态，并取得控制。为了能继续执行这个进程，操作系统要把包含引发访问故障的逻辑地址的进程块取进内存。为此，操作系统产生一个磁盘 I/O 读请求。产生 I/O 请求后，在执行磁盘 I/O 期间，操作系统可以调度另一个进程运行。一旦需要的块被取进内存，则产生一个 I/O 中断，控制被交回操作系统，而操作系统把由于

缺少该块而被阻塞的进程置回就绪态。

进程在执行过程中仅仅因为没有装入所有需要的进程块而不得被中断,这种方法的效率问题很让人怀疑。现在暂且不考虑保证效率的问题,而先考虑新策略的实现问题。有两种实现方法可以提高系统的利用率,其中第二种的效果比第一种更令人吃惊。这两种实现方法分别是:

1) 在内存中保留多个进程。由于对任何特定的进程都仅仅装入它的某些块,因此就有足够的空间来放置更多的进程。这样,在任何时刻这些进程中都能至少有一个处于就绪状态,于是处理器得到了更有效的利用。

2) 进程可以比内存的全部空间还大。程序占用的内存空间的大小是程序设计中最大的限制之一。如果没有这种方案,程序员必须清楚地知道有多少内存空间可用。如果编写的程序太大,程序员就必须设计出能把程序分成块的方法,这些块可以按某种覆盖策略分别加载。通过基于分页或分段的虚拟内存,这项工作可以由操作系统和硬件完成。对程序员而言,他所处理的是一个巨大的内存,大小与磁盘存储器相关。操作系统在需要时,自动把进程块装入内存。

由于一个进程只能在内存中执行,因此这个存储器称做实存储器(real memory),简称实存。但是程序员或用户感觉到的是一个更大的内存,通常它被分配在磁盘上,这称为虚拟内存(virtual memory),简称虚存。虚存允许更有效的多道程序设计,并解除了用户与内存之间没有必要的紧密约束。表 8.2 总结了使用虚存和不使用虚存的情况下分页和分段的特点。

表 8.2 分页和分段的特点

简单分页	虚存分页	简单分段	虚存分段
内存被划分成大小固定的小块,称做页框	内存被划分成大小固定的小块,称做页框	内存未被划分	内存未被划分
程序被编译器或内存管理系统划分成页	程序被编译器或内存管理系统划分成页	由程序员给编译器指定程序段(也就是说,由程序员决定)	由程序员给编译器指定程序段(也就是由程序员决定)
页框中有内部碎片	页框中有内部碎片	没有内部碎片	没有内部碎片
没有外部碎片	没有外部碎片	有外部碎片	有外部碎片
操作系统必须为每个进程维护一个页表,以说明每个页对应的页框	操作系统必须为每个进程维护一个页表,以说明每个页对应的页框	操作系统必须为每个进程维护一个段表,以说明每一段中的加载地址和长度	操作系统必须为每个进程维护一个段表,以说明每一段中的加载地址和长度
操作系统必须维护一个空闲页框列表	操作系统必须维护一个空闲页框列表	操作系统必须维护一个内存中的空闲的空洞列表	操作系统必须维护一个内存中的空闲的空洞列表
处理器使用页号和偏移量来计算绝对地址	处理器使用页号和偏移量来计算绝对地址	处理器使用段号和偏移量来计算绝对地址	处理器使用段号和偏移量来计算绝对地址
当进程运行时,它的所有页必须都在内存中,除非使用了覆盖技术	当进程在运行时,并不是它的所有页都必须在内存页框中。只在需要时才读入页	当进程在运行时,它的所有段都必须在内存中,除非使用了覆盖技术	当进程在运行时,并不是它的所有段都必须在内存页框中。只在需要时才读入段
	把一页读入内存可能需要把另一页写到磁盘		把一段读入内存可能需要把另外的一个段或几个段写出到磁盘

8.1.1 局部性和虚拟内存

虚拟内存的优点是很具有吸引力的,但这个方案切实可行吗?关于这一点曾经有过相当多的争论,但是众多操作系统中的经验已多次证明了虚拟内存的可行性。因此,基于分页或者分页和分段的虚拟内存已经成为当代操作系统的一个基本构件。

为理解关键问题是什么以及为什么会有这么多的争论,需要再次分析一下就虚拟内存而言的操作系统任务。考虑一个由很长的程序和许多个数组的数据组成的大进程。在任何一段很短的

时间内, 执行可能会局限在很小的一段程序中 (如一个子程序), 并且可能仅仅会访问一个或两个数组的数据。这样, 如果在程序被挂起或被换出前仅仅使用了一部分进程块, 那么为该进程给内存中装入太多的块显然会带来巨大的浪费。可以通过仅仅装入这一小部分块来更好地使用内存。然后, 如果程序转移到或访问到不在内存中的某个块中的指令或数据, 就会引发一个错误, 告诉操作系统读取需要的块。

因此, 在任何时刻, 任何一个进程只有一部分块位于内存中, 可以在内存中保留更多的进程。此外, 由于未用到的块不需要换入换出内存, 因而节省了时间。但是, 操作系统必须很“聪明”地管理这个方案。在稳定状态, 几乎内存的所有空间都被进程块占据, 处理器和操作系统可以直接访问到尽可能多的进程。因此, 当操作系统读取一块时, 它必须把另一块换出。如果一块正好在将要被用到之前换出, 操作系统就不得不很快把它取回来。太多的这类操作会导致一种称为系统抖动 (thrashing) 的情况: 处理器的大部分时间都用于交换块, 而不是执行指令。在 20 世纪 70 年代, 如何避免系统抖动是一个重要的研究领域, 同时也出现了许多复杂但有效的算法。从本质上看, 这些算法都是操作系统试图根据最近的历史来猜测在不远的将来最可能用到的块。

这类推断基于局部性原理 (principle of locality), 局部性原理在本书第 1 章曾经介绍过 (参见附录 1A)。概括来说, 局部性原理描述了一个进程中程序和数据引用的集簇倾向。因此, 假设在很短的时间内仅需要进程的一部分块是合理的。同时, 还可以对在不远的将来可能会访问的块进行猜测, 从而避免系统抖动。

证实局部性原理的一种方法是在虚拟内存环境中查看进程的执行情况。图 8.1 是一个动态地阐明了局部性原理的著名图表 [HATF72]。注意, 在进程的生命周期中, 所有引用都局限在进程页的一个子集中。

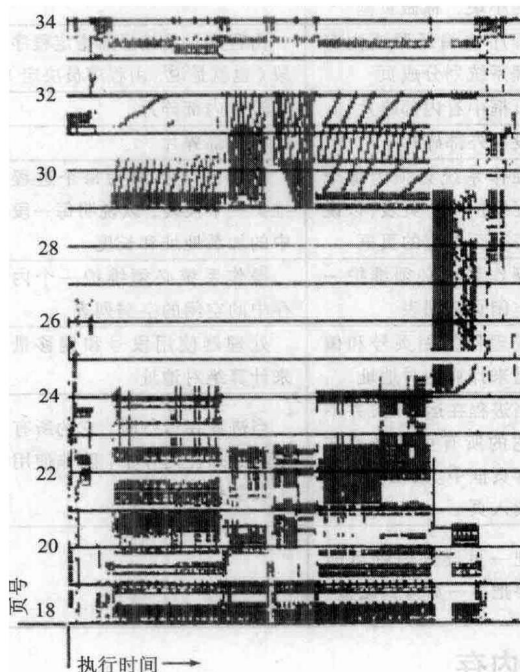


图 8.1 分页下的运行情况

局部性原理说明了虚拟内存方案是可行的。为了使虚拟内存比较实用并且有效, 需要两方面的因素。首先, 必须有对所采用的分页或分段方案的硬件支持; 其次, 操作系统必须有管理页或段在内存和辅助存储器 (简称辅存) 之间移动的软件。本节首先分析硬件特征, 然后介绍由操作

系统创建并维护、供内存管理硬件使用的必要的控制结构。下一节将介绍操作系统方面的问题。

8.1.2 分页

虽然存在基于分段的虚拟内存（接下来会讲述），术语虚拟内存通常与使用分页的系统联系在一起。第一个使用分页实现虚拟内存的是 Atlas 计算机[KILB62]，随后很快广泛应用于商业用途。

在讲述简单分页时，曾指出每个进程都有自己的页表，当它的所有页都装入到内存中时，页表被创建并被装入内存。页表项（Page Table Entry，简称 PTE）包含有与内存中的页框相对应的页框号。当考虑基于分页的虚拟内存方案时也同样需要页表，并且通常每个进程都有一个唯一的页表，但这时页表项变得更复杂，如图 8.2a 所示。由于一个进程可能只有一些页在内存中，因而每个页表项需要有一位（P）来表示它所对应的页当前是否在内存中。如果这一位表示该页在内存中，则这个页表项还包括该页的页框号。

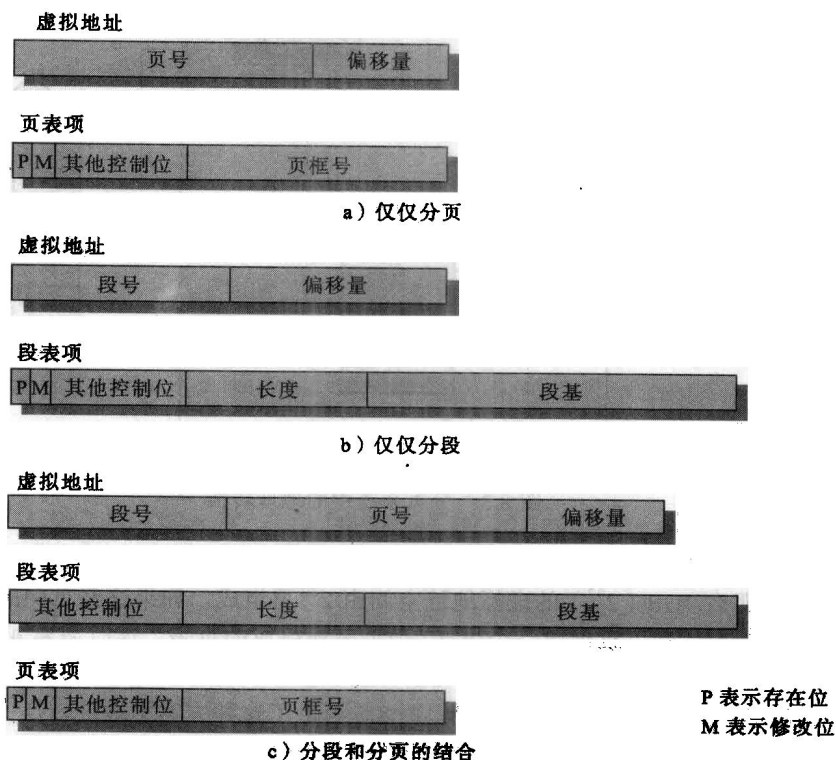


图 8.2 典型的内存管理格式

页表项中所需要的另一个控制位是修改位（M），表示相应页的内容从上一次装入内存中到现在是否已经改变。如果没有改变，则当需要把该页换出时，不需要用页框中的内容更新该页。还必须提供其他一些控制位，例如，如果需要在页一级控制保护或共享，则需要用于这些目的位。

页表结构

从存储器中读取一个字的基本机制包括使用页表从虚拟地址到物理地址的转换。虚拟地址又称为逻辑地址，由页号和偏移量组成，而物理地址由页框号和偏移量组成。由于页表的长度可以基于进程的长度而变化，因而不能期望在寄存器中保存它，它必须在内存中且可以访问到。图 8.3 给出了一种硬件实现。当一个特定的进程正在运行时，一个寄存器保存该进程页表的起始地址。虚拟地址的页号用于检索页表、查找相应的页框号，并与虚拟地址的偏移量组合起来产生需

要的实地址。一般来说，页号域长于页框号域 ($n > m$)。

在大多数系统中，每个进程都有一个页表。但是每个进程可以占据大量的虚拟内存空间。例如，在 VAX 系统结构中，每个进程可以有接近 $2^{31} = 2\text{GB}$ 的虚拟内存空间，如果使用 $2^9 = 512$ 字节的页，这意味着每个进程需要有 2^{22} 个页表项。显然，采用这种方法用于放置页表的内存空间实在太多了。为了克服这个问题，大多数虚拟内存方案都在虚拟内存中而不是在实内存中保存页表。这意味着页表和其他页一样都服从分页管理。当一个进程正在运行时，它的页表至少有一部分必须在内存中，这一部分包括正在运行的页的页表项。一些处理器使用两级方案组织大型页表。在这类方案中有一个页目录，每一项指向一个页表，因此，如果页目录的长度为 X ，并且如果一个页表的最大长度为 Y ，一个进程可以有 $X \times Y$ 页。在典型情况下，一个页表的最大长度被限制为一页。例如，Pentium 处理器就使用了这种方法。

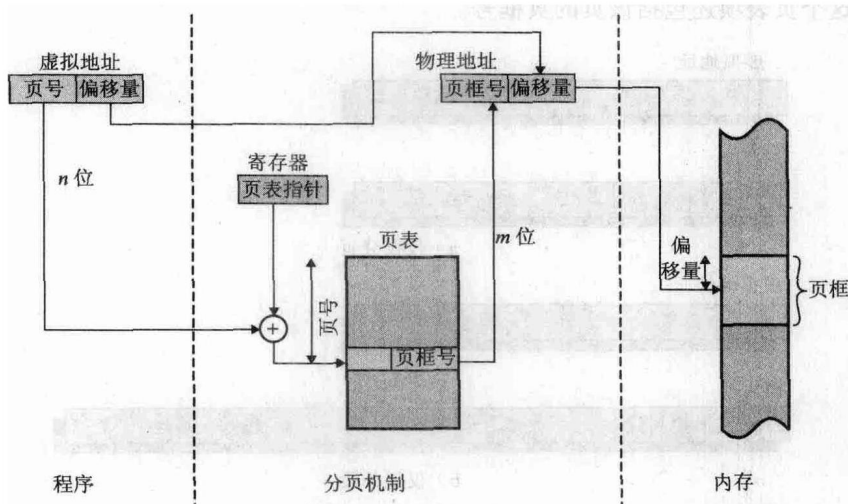


图 8.3 分页系统中的地址转换

图 8.4 给出了一个用于 32 位地址的两级方案的典型例子。假设采用字节级的寻址，页尺寸为 4KB (2^{12})，那么 4GB (2^{32}) 的虚拟地址空间由 2^{20} 页组成。如果这些页中的每一个都由一个 4 字节的页表项映射，则可以创建一个由 2^{20} 个页表项组成的页表，这时需要 4MB (2^{22}) 内存空间。这个由 2^{10} 页组成的巨大的用户页表可以保留在虚拟内存中，由一个包括 2^{10} 个页表项的根页表映射，根页表占据 4KB (2^{12}) 的内存。图 8.5 给出了这种方案中地址转换所涉及的步骤。虚拟地址的前 10 位用于检索根页表，查找关于用户页表的页的页表项。如果该页不在内存中，则发生一次缺页中断。如果该页在内存中，则用虚拟地址中接下来的 10 位检索用户页表项，查找该虚拟地址引用的页的页表项。

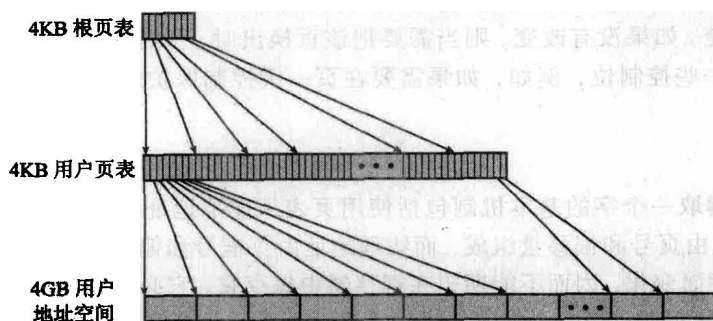


图 8.4 两级层次页表

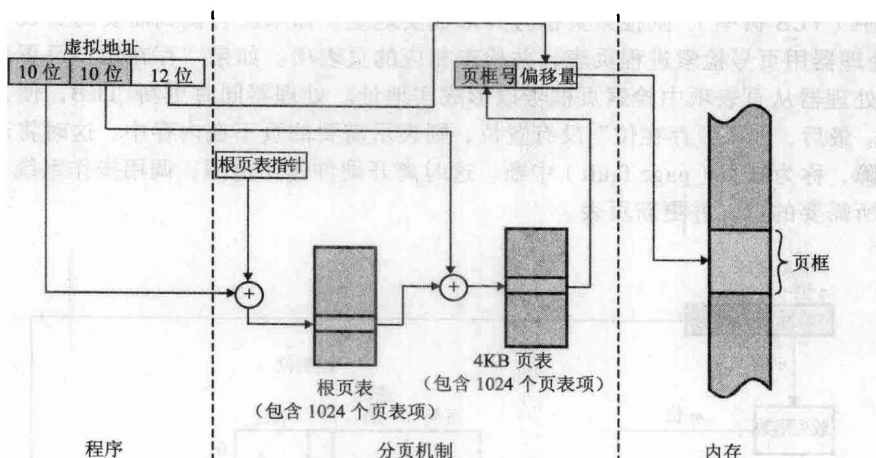


图 8.5 两级分页系统中的地址转换

倒排页表

前面讨论的页表设计的一个重要缺陷是页表的大小与虚拟地址空间的大小成正比。

使用一级或多级页表的一种替代方法是使用一个倒排页表结构，该方法的各种变种用于 PowerPC、UltraSPARC 和 IA-64 体系结构中，RT-PC 上的 Mach 操作系统的实现也使用了这种技术。

在这种方法中，虚拟地址的页号部分使用一个简单的散列函数映射到散列表中^①。散列表包含一个指向倒排表的指针，而倒排表中含有页表项。通过这个结构，散列表和倒排表中各有一项对应于一个实存页，而不是虚拟页。因此，不论有多少进程、支持多少虚拟页，页表都只需要实存中的一个固定部分。由于多个虚拟地址可能映射到同一个散列表项中，因此需要使用一种链接技术管理这种溢出。散列技术使得链一般都比较短，通常只有一到两项。页表的结构称为“倒排”是因为它使用页框号而不是虚拟页号来索引页表项。

图 8.6 说明了一个倒排页表方法的典型实现。对于大小为 2^m 个页框的物理内存，倒排页表包含 2^m 项，所以第 i 个项对应第 i 个页框。页表中的每项都包含如下内容：

- 页号：虚拟地址的页号部分。
- 进程标志符：使用该页的进程。页号和进程标志符结合起来标志一个特定进程的虚拟地址空间的一页。
- 控制位：该域包含一些标记，比如有效、访问和修改；以及保护和锁定信息。
- 链指针：如果某个项没有链项，则该域为空（或许用一个单独的位来表示）。否则，该域包含链中下一项的索引值（在 0 到 2^m-1 之间的数字）。

在图 8.6 的例子中，虚拟地址包含一个 n 位的页号，并且 $n > m$ 。散列函数映射 n 位页号到 m 位数，这个 m 位数用于索引倒排页表。

转换检测缓冲区

原则上，每个虚存访问可能引起两次物理内存访问：一次取相应的页表项，一次取需要的数据。因此，简单的虚拟内存方案会导致存储器访问时间加倍。为克服这个问题，大多数虚拟内存方案为页表项使用一个特殊的高速缓存，通常称做转换检测缓冲区（Translation Lookaside Buffer, TLB）。这个高速缓存的功能和高速缓冲存储器（见第 1 章）相似，包含最近用过的页表项。由此得到的分页硬件组织如图 8.7 所示。给定一个虚拟地址，处理器首先检查 TLB，如果需要的页

① 见附录 8A 中关于散列技术的讨论。

表项在其中 (TLB 命中), 则检索页框号并形成实地址。如果没有找到需要的页表项 (TLB 未命中), 则处理器用页号检索进程页表, 并检查相应的页表项。如果“存在位”已置位, 则该页在内存中, 处理器从页表项中检索页框号以形成实地址。处理器同时更新 TLB, 使其包含这个新的页表项。最后, 如果“存在位”没有置位, 则表示需要的页不在内存中, 这时将产生一次存储器访问故障, 称为缺页 (page fault) 中断。这时离开硬件作用范围, 调用操作系统, 由操作系统负责装入所需要的页, 并更新页表。

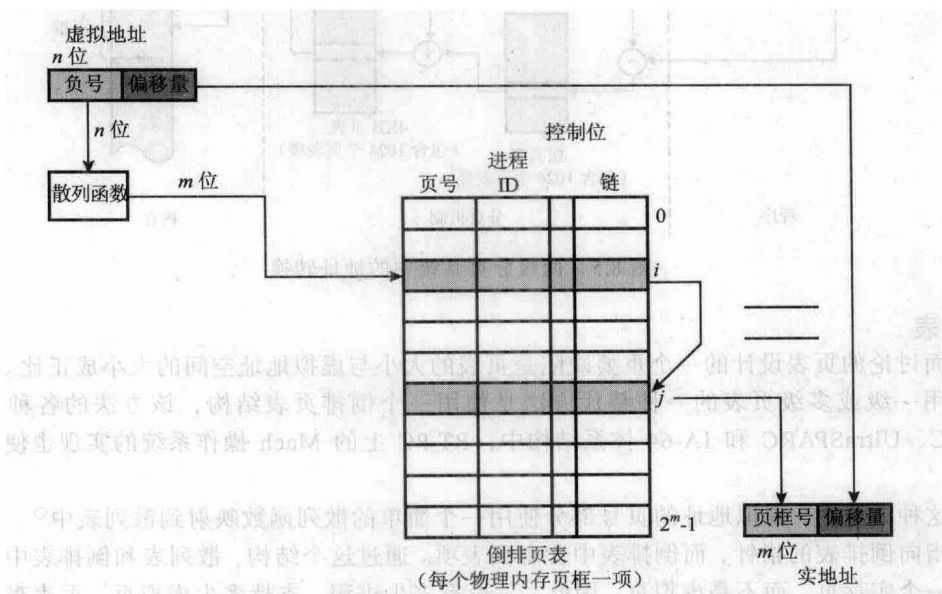


图 8.6 倒排页表结构

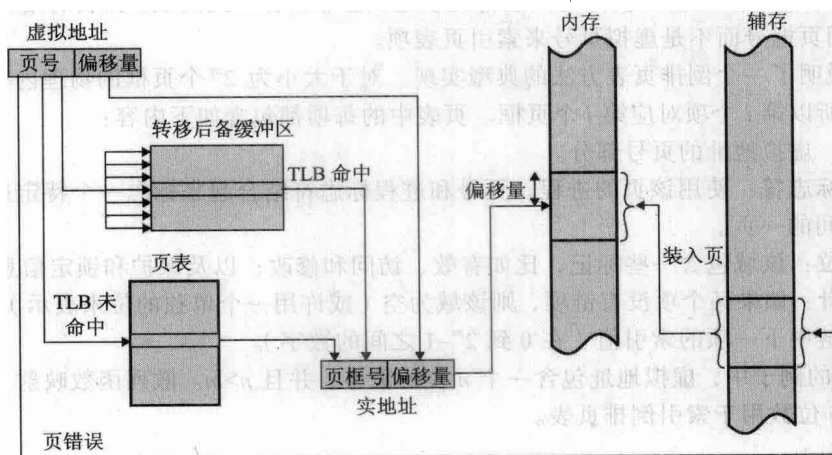


图 8.7 转换检测缓冲区的用法

图 8.8 中的流程图表明了 TLB 的使用。如果需要的页不在内存中, 一个缺页中断导致调用缺页中断处理例程。为保持流程图简洁, 图中没有表明在磁盘 I/O 过程中操作系统可以分派另一个进程执行。根据局部性原理, 大多数虚拟内存访问都位于最近使用过的页中, 因此, 大多数访问将调用高速缓存中的页表项。针对 VAX TLB 的研究表明, 该方案可以较大地提高性能 [CLAR85, SATY81]。

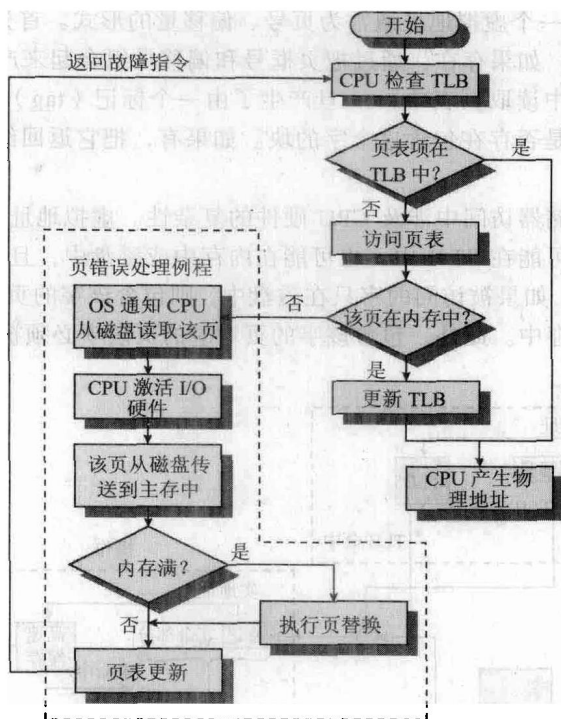


图 8.8 分页和转移检测缓冲区 (TLB) 的操作 [FURH87]

关于 TLB 的实际组织还有很多另外的细节问题。由于 TLB 仅包含整个页表中的部分表项，所以不能简单地把页号编入 TLB 的索引。相反，TLB 中的项必须包括页号以及完整的页表项。处理器中的硬件机制允许同时查询许多 TLB 页，以确定是否存在匹配的页号。对应于图 8.9 中在页表中查找所使用的直接映射或索引，该技术称为关联映射 (associative mapping)。TLB 的设计还必须考虑 TLB 中表项的组织方法，以及当读取一个新项时置换哪一项。这些问题是任何硬件高速缓存设计中都必须考虑的，这里不再继续讨论，详细信息请参阅高速缓存设计方面的资料 (例如 [STAL06a])。

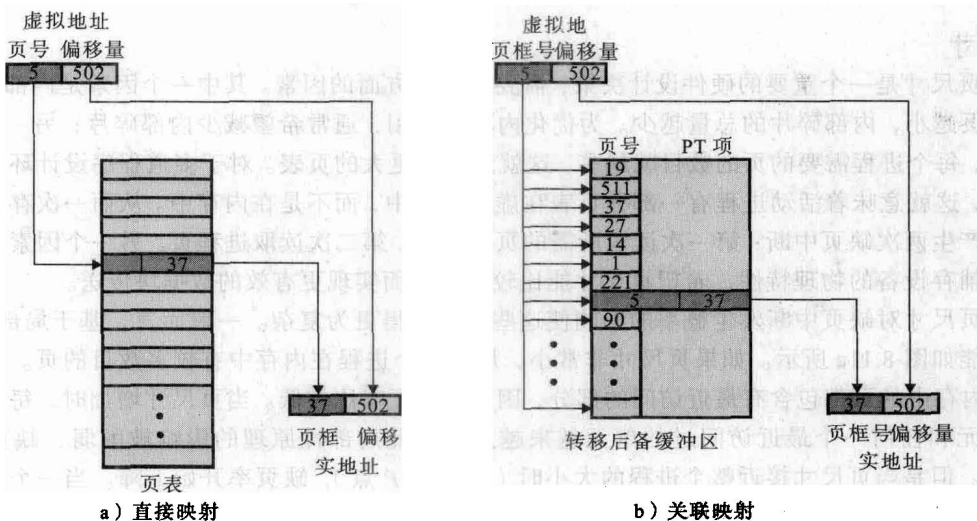


图 8.9 页表项的直接查找和关联查找

最后，虚拟内存机制必须与高速缓存系统 (不是 TLB 高速缓存，而是内存高速缓存) 进行

交互，如图 8.10 所示。一个虚拟地址通常为页号、偏移量的形式。首先，内存系统查看 TLB 中是否存在匹配的页表项，如果存在，通过把页框号和偏移量组合起来产生实地址（物理地址）；如果不存在，则从页表中读取页表项。一旦产生了由一个标记（tag）^①和其余部分组成的实地址，则查看高速缓存中是否存在包含这个字的块。如果有，把它返回给 CPU；如果没有，从内存中检索这个字。

需要注意在一次存储器访问中涉及 CPU 硬件的复杂性。虚拟地址被转换成实地址，这涉及访问页表项，而页表项可能在 TLB 中，也可能在内存中或磁盘上，且被访问的字可能在高速缓存中、内存中或磁盘上。如果被访问的字只在磁盘上，则包含该字的页必须装入内存中，并且它所在的块装入到高速缓存中。此外，包含该字的页对应的页表项必须被更新。

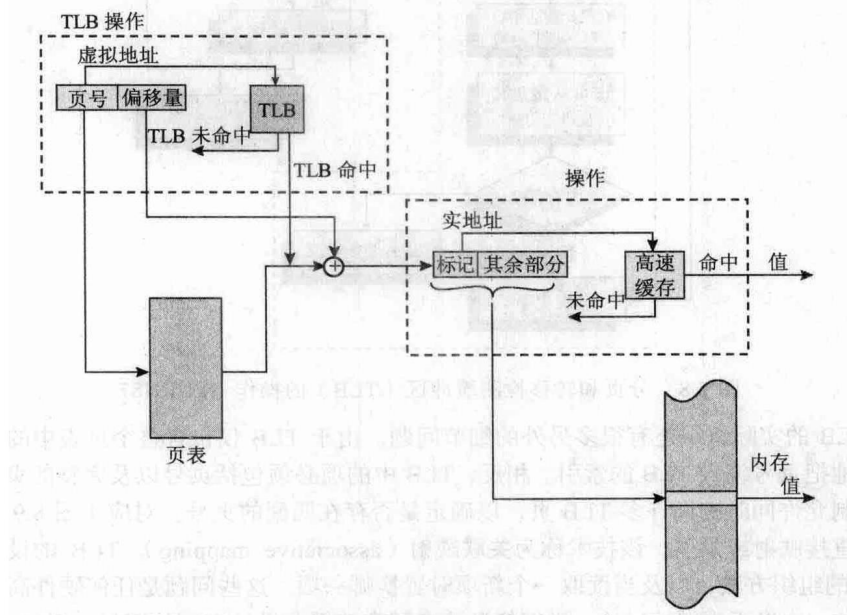


图 8.10 转换检测缓冲区和高速缓存操作

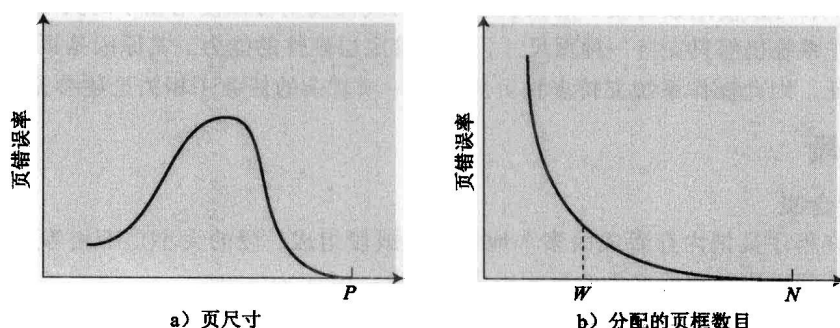
页尺寸

页尺寸是一个重要的硬件设计决策，需要考虑多方面的因素。其中一个因素是内部碎片。显然，页越小，内部碎片的总量越少。为优化内存的使用，通常希望减少内部碎片；另一方面，页越小，每个进程需要的页的数目就越多，这就意味着更大的页表。对于多道程序设计环境中的大程序，这就意味着活动进程有一部分页表在虚拟内存中，而不是在内存中。从而一次存储器访问可能产生两次缺页中断：第一次读取所需的页表部分，第二次读取进程页。另一个因素是基于大多数辅存设备的物理特性，希望页尺寸能比较大，从而实现更有效的数据块传送。

页尺寸对缺页中断发生概率的影响使这些问题变得更为复杂。一般而言，基于局部性原理，其性能如图 8.11a 所示。如果页尺寸非常小，那么每个进程在内存中有较多数目的页。一段时间后，内存中的页都包含有最近访问的部分，因此，缺页率比较低。当页尺寸增加时，每一页包含的单元和任何一个最近访问过的单元越来越远。因此局部性原理的影响被削弱，缺页率开始增长。但是当页尺寸接近整个进程的大小时（图示的 P 点），缺页率开始下降。当一个页包含整个进程时，不会发生缺页中断。

① 参见图 1.17。一般来说，标记只是实地址最左边的位。关于高速缓存的更多讨论，请参考[STAL06a]。

更为复杂的是, 缺页率还取决于分配给一个进程的页框的数目。图 8.11b 表明, 对固定的页尺寸, 当内存中的页数增加时, 缺页率会下降^①。因此, 软件策略 (分配给每个进程的内存总量) 影响着硬件设计决策 (页尺寸)。



P 表示整个进程的大小, W 表示工作集大小, N 表示进程中的总页数

图 8.11 典型的分页行为

表 8.3 给出了大多数机器中采用的页尺寸。

最后, 页尺寸的设计问题与物理内存的大小和程序大小有关。当内存变大时, 应用程序使用的地址空间也相应地增长, 这种趋势在个人计算机和工作站上更为显著。此外, 大型程序中所使用的当代程序设计技术可能会降低进程中的局部性 [HUCK93]。例如:

- 面向对象技术鼓励使用小程序和数据模块, 关于它们的引用在相对比较短的时间里散布在相对比较多的对象中。
- 多线程应用可能导致指令流和分散的存储器访问的突然变化。

表 8.3 页尺寸的例子

计 算 机	页 尺 寸
Atlas	512 个 48 位字
Honeywell-Multics	1024 个 36 位字
IBM 370/XA 和 370/ESA	4KB
IBM AS/400	512 字节
DEC Alpha	8KB
MIPS	4KB ~ 16MB
UltraSPARC	8KB ~ 4MB
Pentium	4KB ~ 4MB
IBM POWER	4KB
Itanium	4KB ~ 256MB

对于给定大小的 TLB, 当进程的内存大小增加并且局部性降低时, TLB 访问的命中率降低。在这种情况下, TLB 可能成为一个性能瓶颈 (例如, 见 [CHEN92])。

提高 TLB 性能的一种方法是使用包含更多项的更大的 TLB。但是, TLB 的大小会影响其他的硬件设计特征, 如内存高速缓存和每个指令周期访问内存的数量 [TALL92], 因此 TLB 的大小不可能像内存大小增长得那么快。一种可选的方法是采用更大的页, 使得 TLB 中的每个页表项对应于更大的存储块。但由前面的讨论得知, 采用较大的页可能导致性能下降。

① 变量 W 代表工作集的大小, 其概念将在 8.2 节讨论。

因此,很多硬件设计者都尝试使用多种页大小 [TALL92, KHAL93],并且很多微处理器体系结构支持多种页尺寸,包括 MIPS R4000、Alpha、UltraSPARC、Pentium 和 IA-64 等。多种页尺寸为有效地使用 TLB 提供了很大的灵活性。例如,一个进程的地址空间中一大片连续的区域(如程序指令),可以使用数目较少的大页映射,而线程栈则可以使用较小的页来映射。但是,大多数商业操作系统仍然只支持一种页尺寸,而不管底层硬件的能力。其原因是页尺寸影响操作系统的许多特征,因此操作系统支持多种页尺寸是一项复杂的任务(相关论述参见 [GANA98])。

8.1.3 分段

虚拟内存的含义

分段允许程序员把内存看成由多个地址空间或段组成,段的大小是不相等的,并且是动态的。存储器访问以段号和偏移量的形式组成的地址。

对程序员而言,这种组织与非段式地址空间相比有许多优点:

- 1) 简化对不断增长的数据结构的处理。如果程序员事先不知道一个特定的数据结构会变得多大,除非允许使用动态的段大小,否则必须对其大小进行猜测。而对于段式虚拟内存,这个数据结构可以分配到它自己的段,需要时操作系统可以扩大或缩小这个段。如果需要被扩大的段在内存中,并且内存中已经没有足够的空间,操作系统可能把这个段移到内存中的一个更大的区域(如果可以得到),或者把它换出。对于后一种情况,被扩大的段将在下一次有机会时被换回。
- 2) 允许程序独立地改变或重新编译,而不要求整个程序集合重新链接和重新加载。同样,这也是使用多个段实现的。
- 3) 有助于进程间的共享。程序员可以在段中放置一个实用工具程序或一个有用的数据表,供其他进程访问。
- 4) 有助于保护。由于一个段可以被构造成包含一个明确定义的程序或数据集,因而程序员或系统管理员可以更方便地指定访问权限。

组织

在讨论简单分段时,曾指出每个进程都有自己的段表,当它的所有段都装入内存时,为该进程创建一个段表并装入内存。每个段表项包含相应段在内存中的起始地址和段的长度。基于分段的虚拟内存方案仍然需要段表这个设计,并且每个进程都有一个唯一的段表。在这种情况下,段表项变得更加复杂,如图 8.2b 所示。由于一个进程可能只有一部分段在内存中,因而每个段表项中需要有一位表明相应的段是否在内存中。如果这一位表明该段在内存中,则这个表项还包括该段的起始地址和长度。

段表项中需要的另一个控制位是修改位,用于表明相应的段从上一次被装入内存到目前为止其内容是否被改变。如果没有改变,把该段换出时就不需要写回。同时还可能需要其他的控制位,例如若要在段级来管理保护或共享,则需要具有用于这种目的的位。

从存储器中读一个字的基本机制涉及使用段表来将段号和偏移量组成的虚拟地址(或逻辑地址)转换为物理地址。根据进程的大小,段表长度可变,而无法在寄存器中保存,因此访问段表时它必须在内存中。图 8.12 表明了该方案的一种硬件实现(与图 8.3 类似)。当一个特定的进程正在运行时,有一个寄存器为该进程保存段表的起始地址。虚拟地址中的段号用于检索这个表,并查找该段起点的相应内存地址。这个地址加上虚拟地址中的偏移量部分,产生了需要的实地址。

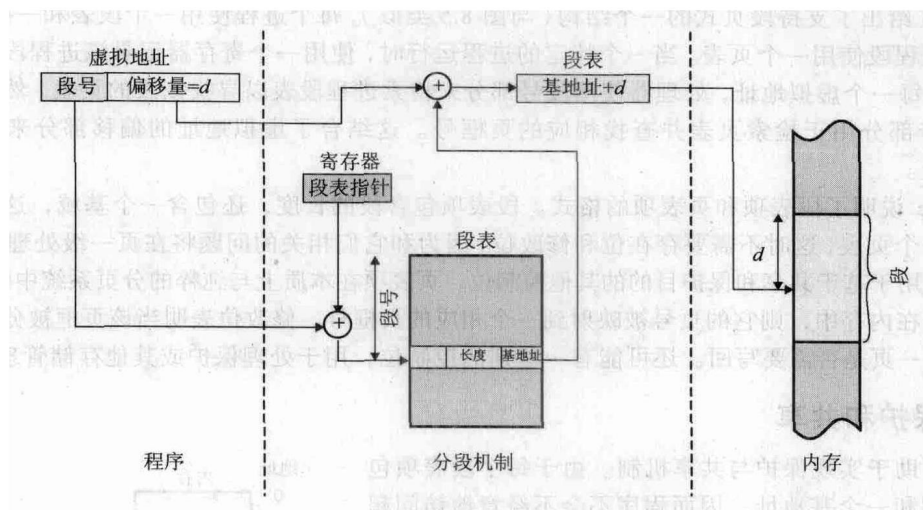


图 8.12 分段系统中的地址转换

8.1.4 段页式

分页和分段都有它们的长处。分页对程序员是透明的，它消除了外部碎片，因而可以更有效地使用内存。此外，由于移入或移出内存的块是固定的、大小相等的，因而有可能开发出更精致的存储管理算法。分段对程序员是可见的，它具有处理不断增长的数据结构的能力以及支持共享和保护的能力。为了把它们二者的优点结合起来，一些系统配备了特殊的处理器硬件和操作系统软件来同时支持这两者。

在段页式的系统中，用户的地址空间被程序员划分成许多段。每个段依次划分成许多固定大小的页，页的长度等于内存中的页框大小。如果某一段的长度小于一页，则该段只占据一页。从程序员的角度看，逻辑地址仍然由段号和段偏移量组成；从系统的角度看，段偏移量可看做是指定段中的一个页号和页偏移量。

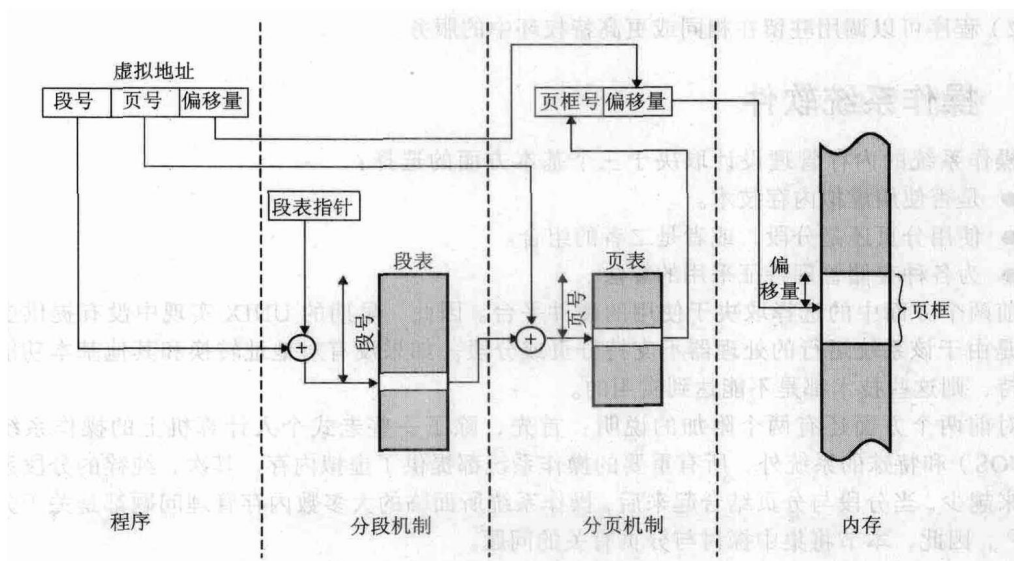


图 8.13 段页式系统中的地址转换

图 8.13 给出了支持段页式的一个结构(与图 8.5 类似)。每个进程使用一个段表和一些页表,并且每个进程段使用一个页表。当一个特定的进程运行时,使用一个寄存器记录该进程段表的起始地址。对每一个虚拟地址,处理器使用段号部分来检索进程段表以寻找该段的页表。然后虚拟地址的页号部分用于检索页表并查找相应的页框号。这结合了虚拟地址的偏移部分来产生需要的实地址。

图 8.2c 说明了段表项和页表项的格式。段表项包含段的长度,还包含一个基域,这个基域现在指向一个页表,这时不需要存在位和修改位,因为它们相关的问题将在页一级处理。此外,还可能需要用于基于共享和保护目的的其他控制位。页表项在本质上与纯粹的分页系统中的相同,如果某一页在内存中,则它的页号被映射到一个相应的页框号。修改位表明当该页框被分配给其他页时,这一页是否需要写回。还可能有一些别的控制位,用于处理保护或其他存储管理特征。

8.1.5 保护和共享

分段有助于实现保护与共享机制。由于每个段表项包括一个长度和一个基地址,因而程序不会不经意地访问超出该段的内存单元。为实现共享,一个段可能在多个进程的段表中被引用。当然,在分页系统中也可以得到同样的机制。但是,此种情况下程序的页结构 and 数据对程序员不可见,使得共享和保护的要求难以说明。图 8.14 说明了这类系统中可以实施的保护关系的类型。

同时也存在更高级的机制,一个常用的方案是使用环状保护结构(参见第 3 章图 3.18)。在这个方案中,编号小的内环比编号大的外环具有更大的特权。在典型情况下,0 号环为操作系统的内核函数保留,应用程序则位于更高层的环。一些实用工具程序或操作系统服务可能占据了中间的环。环状系统的基本原理如下:

- 1) 程序可以只访问驻留在同一个环或更低特权环中的数据。
- 2) 程序可以调用驻留在相同或更高特权环中的服务。

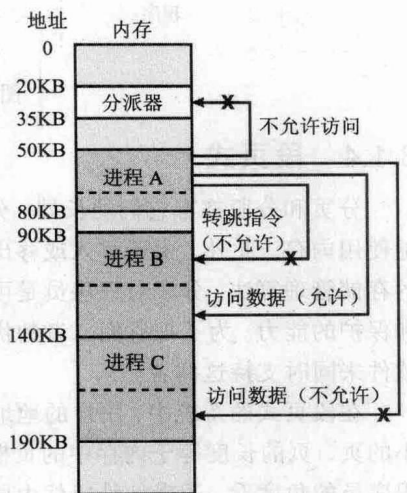


图 8.14 段之间的保护关系

8.2 操作系统软件

操作系统的内存管理设计取决于三个基本方面的选择:

- 是否使用虚拟内存技术。
- 使用分页还是分段,或者是二者的组合。
- 为各种存储管理特征采用的算法。

前两个方面中的选择取决于使用的硬件平台。因此,早期的 UNIX 实现中没有提供虚拟内存,是由于该系统运行的处理器不支持分页或分段。如果没有对地址转换和其他基本功能的硬件支持,则这些技术都是不能达到实用的。

对前两个方面还有两个附加的说明:首先,除了一些老式个人计算机上的操作系统(如 MS-DOS)和特殊的系统外,所有重要的操作系统都提供了虚拟内存。其次,纯粹的分段系统现在越来越少,当分段与分页结合起来后,操作系统所面临的大多数内存管理问题都是关于分页方面的^①。因此,本节将集中探讨与分页有关的问题。

① 在段页式的系统中,保护和共享通常在段级进行处理。这些话将在后面的章节中讨论。

与第三方面相关的选择是属于操作系统软件领域的问题，也是本节的主题。表 8.4 列出了需要考虑的重要的设计因素。在各种情况下，最重要的都是与性能相关的问题：由于缺页中断带来巨大的软件开销，所以希望使缺页中断发生的频率最小。这类开销至少包括决定置换哪个或哪些驻留页以及交换这些页所需要的 I/O 操作。此外，在这个页 I/O 操作的过程中，操作系统还必须调度另一个进程运行，即导致一次进程切换。因此，希望能通过适当的安排，使得在一个进程正在执行时，访问一个未命中的页中的字的可能性最小。在表 8.4 中给出的所有策略中，都不存在一种绝对的最佳策略。在分页环境中的内存管理任务是极其复杂的。此外，任何特定策略的总性能取决于内存的大小、内存和外存的相对速度、竞争资源的进程大小和数目以及单个程序的执行情况。最后一个特性取决于应用程序的类型、所采用的程序设计语言和编译器、编写该程序的程序员的风格和用户的动态行为（交互式程序）。因此，不要期望在本书或者在任何地方会有一个最终答案。对于小系统，操作系统设计者可以试图基于当前的状态信息，选择一组看上去在大多数条件下都比较“好”的策略；而对于大系统，特别是大型机，操作系统应该配备监视和控制工具，允许系统管理员根据系统状态调整操作系统，以获得比较“好”的结果。

表 8.4 用于虚拟内存的操作系统策略

读取策略	驻留集管理
请求分页	驻留集合大小
预先分页	固定
放置策略	可变
置换策略	置换范围
基本算法	全局
最优	局部
最近最少使用算法（LRU）	清除策略
先进先出算法（FIFO）	请求式清除
时钟	预约式清除 时钟
页缓冲	装入控制
	系统并发度

8.2.1 读取策略

读取策略确定一个页何时取入内存，常用的两种方法是请求分页（demand paging）和预先分页（prepaging）。对于请求分页，只有当访问到某页中的一个单元时才将该页取入内存。如果内存管理的其他策略比较合适，将发生下述情况：当一个进程第一次启动时，会在一段时间出现大量的缺页中断；当越来越多的页被取入后，局部性原理表明大多数将来访问的页都是最近读取的页。因此，在一段时间后错误会逐渐减少，缺页中断的数目会降到很低。

对于预先分页，读取的页并不是缺页中断请求的页。预先分页利用了大多数辅存设备（如磁盘）的特性，这些设备有寻道时间和合理的延迟。如果一个进程的页被连续存储在辅存中，则一次读取许多连续的页比隔一段时间读取一页要更有效。当然，如果大多数额外读取的页没有引用到，则这个策略是低效的。

当进程第一次启动时，可以采用预先分页策略，在这种情况下，程序员必须以某种方式指定需要的页；当发生缺页中断时也可以采用预先分页策略，由于这个过程对程序员是不可见的，因而它显得更可取一些。但是，预先分页的实用工具程序还没有建立 [MAEK87]。

不要把预先分页和交换混淆。当一个进程被换出内存并且被置于挂起状态时，它的所有驻留页都被换出。当该进程被唤醒时，所有以前在内存中的页都被重新读回内存。

8.2.2 放置策略

放置策略决定一个进程块驻留在实存中的什么地方。在一个纯粹的分段系统中，放置策略并不是重要的设计问题，第7章讲述的诸如最佳适配、首次适配等都可供选择。但对于纯粹的分页系统或段页式的系统，如何放置通常是没有关系的，这是因为地址转换硬件和内存访问硬件可以以相同的效率为任何页框组合执行它们的功能。

还有一个关注放置问题的领域是非一致存储访问（NonUniform Memory Access, NUMA）多处理器。在非一致存储访问多处理器中，机器中分布的共享内存可以被该机器的任何处理器访问，但是访问某一特定的物理单元所需要的时间随着处理器和内存模块之间距离的不同而变化。因此，其性能很大程度上依赖于数据驻留的位置与使用数据的处理器间的距离 [LAR092, BOLO89, COX89]。对于 NUMA 系统，自动放置策略希望能把页分配到能够提供最佳性能的内存。

8.2.3 置换策略

在大多数操作系统教材中，有关内存管理的内容都包括题目为“置换策略”的一节，用于处理在必须读取一个新页时，应该置换内存中的哪一页。由于涉及许多概念，阐明这方面的主题有一定的困难：

- 给每个活动进程分配多少页框。
- 计划置换页的集合是局限在那些产生缺页中断的进程，还是所有页框都在内存中的进程。
- 在计划置换的页集中，选择换出哪一个页。

前两个概念称做驻留集管理，其内容将在下一节讨论；术语“置换策略”用于专指第三个概念，本节将讲述这方面的内容。

置换策略在内存管理的各个领域都得到了广泛的研究。当内存中的所有页框都被占据，并且需要读取一个新页以处理一次缺页中断时，置换策略决定当前在内存中的哪个页将被置换。所有策略的目标都是移出最近最不可能访问的页。由于局部性原理，最近的访问历史和最近将要访问的模式间有很大的相关性。因此，大多数策略都基于过去的行为来预测将来的行为。必须折中考虑的是，置换策略设计得越精致、越复杂，实现它的软硬件开销就越大。

页框锁定

在分析各种算法前，需要注意的是关于置换策略的一个约束：内存中的某些页框可能是被锁定的。如果一个页框被锁定时，当前保存在该页框中的页就不能被置换。大部分操作系统内核和重要的控制结构就保存在锁定的页框中，此外，I/O 缓冲区和其他对时间要求严格的区域也可能锁定在内存的页框中。锁定是通过给每个页框关联一个 lock 位实现的，这一位可以包含在页框表和当前的页表中。

基本算法

Animation: Rage Replacement Algorithms

不论采用哪种驻留集管理策略（在下一节讲述），都有一些用于选择置换页的基本算法。在文献中可以查到的置换算法包括：最佳（OPT）、最近最少使用（LRU）、先进先出（FIFO）、时钟。

OPT 策略选择置换下次访问距当前时间最长的那些页，可以看出该算法能导致最少的缺页中断 [BELA66]，但是由于它要求操作系统必须知道将来的事件，显然这是不可能实现的。但是它仍能作为一种标准来衡量其他算法的性能。

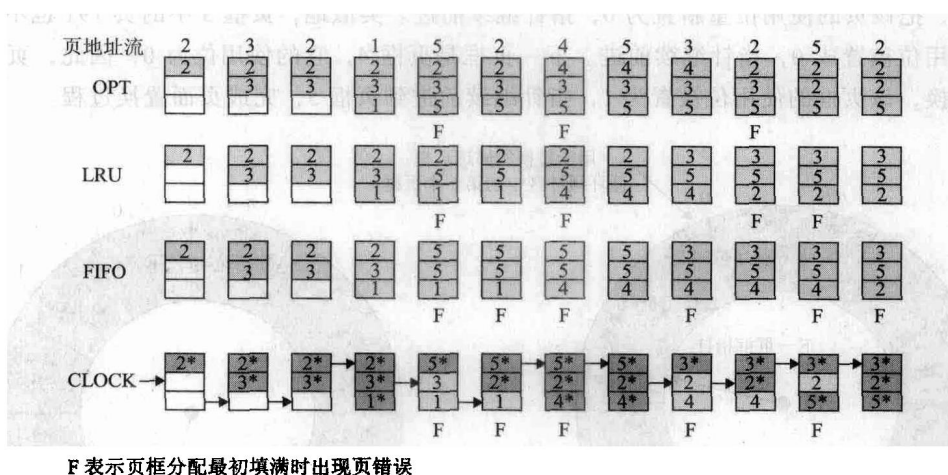
图 8.15 给出了关于 OPT 策略的一个例子, 该例子假设固定的为该进程分配 3 个页框 (驻留集合大小固定)。进程的执行需要访问 5 个不同的页, 运行该程序需要的页地址的顺序为:

2 3 2 1 5 2 4 5 3 2 5 2

这意味着访问的第一个页是 2, 第二个页是 3, 依此类推。当页框分配满了以后, OPT 策略产生 3 次缺页中断。

LRU 策略置换内存中上次使用距当前最远的页。根据局部性原理, 这也是最近最不可能访问到的页。实际上, LRU 策略的性能接近于 OPT 策略。该方法的问题在于比较难于实现。一种实现方法是给每一页添加一个最后一次访问的时间标签, 并且必须在每次访问存储器时, 都更新这个标签。即使有支持这种方案的硬件, 开销仍然是非常大的。另一种可选择的方法是维护一个关于访问页的栈, 但开销同样很大。

图 8.15 给出了关于 LRU 行为的一个例子, 它使用与前面 OPT 策略的例子相同的页地址顺序。在这个例子中会产生 4 次缺页中断。



F 表示页框分配最初填满时出现页错误

图 8.15 4 种页面置换算法的行为

FIFO 策略把分配给进程的页框看做是一个循环缓冲区, 按循环方式移动页。它所需要的只是一个指针, 这个指针在该进程的页框中循环。因此这是一种实现起来最简单的页面置换策略。除了它的简单性, 这种选择方法所隐含的逻辑是置换驻留在内存中时间最长的页: 一个在很久以前取入内存的页, 到现在可能已经不会再用到了。这个推断常常是错误的, 因为经常会出现一部分程序或数据在整个程序的生命周期中使用频率都很高的情况, 如果使用 FIFO 算法, 则这些页会反复地需要被换入换出。

继续图 8.15 中的例子, FIFO 策略导致了 6 次缺页中断。注意到 LRU 识别出页 2 和页 5 比其他页的访问频率高, 而 FIFO 却没有。



Animation: Clock Algorithms

尽管 LRU 策略几乎与 OPT 策略一样好, 但是它的实现比较困难, 而且需要大量的开销。另一方面, FIFO 策略实现简单, 但性能相对较差。这些年以来, 操作系统的设计者尝试了很多其他的算法, 试图以较小的开销接近 LRU 的性能。许多这类算法都是称为时钟策略的各种变体。

最简单的时钟策略需要给每一页框关联一个附加位, 称为使用位。当某一页首次装入内存中时, 则将该页框的使用位设置为 1; 当该页随后被访问到时 (在访问产生缺页中断之后), 它的使用位也会被置为 1。对于页面置换算法, 用于置换的候选页框集合 (当前进程: 局部范围; 整